

Práctica 4: Analizadores sintácticos con Yacc/Bison

Compiladores 2026-2

Profesora: Ariel Adara Mercado Martínez

Alumno

Edwin Rodríguez Nevarez - 312227441

Objetivo

Aprender a definir una gramática en *Yacc/Bison*, Implementar una gramática con *Yacc/Bison*. Unir un programa generado con *Lex/Flex* y el programa generado con *Yacc/Bison*.

Desarrollo

Para resolver los ejercicios del 1 al 4, se tomó el código de ejemplo y se le hicieron las siguientes modificaciones a los archivos `lexer.ll` y `calculadora.yy`:

1. **Operar con flotantes (Ej. 1):** Se modificó la estructura `union` en Bison para usar un `float` en lugar de un `int`. En Flex, se añadió una expresión regular para reconocer puntos decimales (`{DIG}+\.{DIG}+`) y se utilizó `atof()` en lugar de `atoi()`.
2. **Múltiples líneas (Ej. 2):** Se agregó el salto de línea (`\n`) devolviendo un token `EOL`. En la gramática, se cambió el axioma inicial para que acepte una recursión de líneas (`lineas: lineas linea | linea`).
3. **Restas y divisiones (Ej. 3):** Se agregaron los tokens `MENOS` y `DIV`. Se hicieron sus precedencias y asociatividad por la izquierda en Bison, y se hicieron las operaciones en `expresion`, con una validación básica para evitar la división por cero.
4. **Números negativos (Ej. 4):** Se usó el menos unario (`UMINUS`) en Bison. Se hizo su precedencia y se agregó la regla `| MENOS expresion%prec UMINUS { $$fval = -$2.fval; }`.

A continuación, se muestra el código modificado de los archivos:

Archivo modificado: `lexer.ll`

```
1 %{
2 #include <sstream>
3 #include <cstdlib>
4 #include "Parser.hpp"
5 #include "Lexer.hpp"
6 using namespace std;
7
8 #undef YY_DECL
9 #define YY_DECL int calc::Lexer::lex(calc::Parser::semantic_type *yylval)
10 %}
11
```

```

12 %option c++
13 %option noyywrap
14 %option outfile="Lexer.cpp"
15 %option yyclass="Lexer"
16
17 DIG [0-9]
18 WSPC [ \t]+
19
20 %%
21 "+"      { return Parser::token::MAS; }
22 "-"      { return Parser::token::MENOS; }
23 "*"      { return Parser::token::MUL; }
24 "/"      { return Parser::token::DIV; }
25 "("      { return Parser::token::PARIZQ; }
26 ")"      { return Parser::token::PARDER; }
27 "\n"     { return Parser::token::EOL; }
28
29 {DIG}+   {
30     yylval->numero.fval = atof(yytext);
31     yylval->numero.tipo = 1;
32     return Parser::token::NUM;
33 }
34
35 {DIG}+\.{DIG}+ {
36     yylval->numero.fval = atof(yytext);
37     yylval->numero.tipo = 2;
38     return Parser::token::NUM;
39 }
40
41 {WSPC}   { /* Ignoramos espacios en blanco */ }
42
43 .        { cout << "Error lexico en la linea: " << yylineno << endl; }
44
45 <<EOF>> { yyterminate(); }
46 %%

```

Archivo modificado: calculadora.yy

```

1  %{
2  #include <iostream>
3  #include <fstream>
4  #include <string>
5  #include <FlexLexer.h>
6  using namespace std;
7  %}
8
9  %language "C++"
10 %require "3.2"
11 %defines "Parser.hpp"
12 %output "Parser.cpp"
13
14 %define api.parser.class {Parser}
15 %define api.namespace {calc}
16 %parse-param {Lexer* lexer}
17

```

```

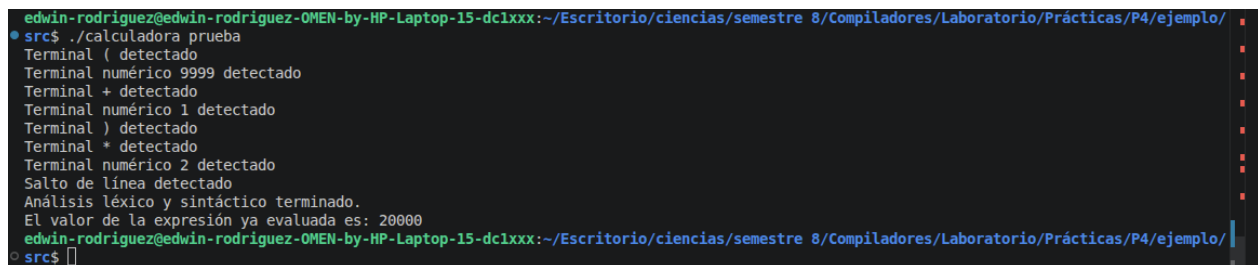
18 %code requires {
19     namespace calc { class Lexer; }
20 }
21
22 %code {
23     #include "Lexer.hpp"
24     #define yylex(x) lexer->lex(x)
25 }
26
27 %union {
28     struct {
29         float fval; // Cambiado a float para soportar decimales
30         int tipo;
31     } numero;
32 }
33
34 %token<numero> NUM
35 %token EOL
36
37 %left MAS MENOS
38 %left MUL DIV
39 %precedence UMINUS // Precedencia para el menos unario
40
41 %type<numero> expresion
42
43 %start lineas
44
45 %%
46
47 lineas: lineas linea
48       | linea
49       ;
50
51 linea: expresion EOL { cout << "El valor de la expresion es: " << $1.fval
52       << endl; }
53       | EOL { /* Ignorar lineas en blanco */ }
54       ;
55
56 expresion : expresion MAS expresion { $$fval = $1.fval + $3.fval; }
57           | expresion MENOS expresion { $$fval = $1.fval - $3.fval; }
58           | expresion MUL expresion { $$fval = $1.fval * $3.fval; }
59           | expresion DIV expresion {
60               if($3.fval == 0) {
61                   cout << "Error matematico: Division por cero." << endl
62                   ;
63                   $$fval = 0;
64               } else {
65                   $$fval = $1.fval / $3.fval;
66               }
67           }
68           | MENOS expresion %prec UMINUS { $$fval = -$2.fval; }
69           | PARIZQ expresion PARDER { $$ = $2; }
70           | NUM { $$ = $1; }
71           ;

```

```

70
71 %%
72
73 void calc::Parser::error(const std::string& msg) {
74     std::cerr << "Error de sintaxis: " << msg << '\n';
75 }
76
77 #include <iostream>
78 #include <string>
79 #include <fstream>
80
81 int main(int argc, char *argv[]){
82     if(argc < 2){
83         cout<<"Faltan argumentos. Uso: ./calculadora archivo_prueba"<<endl
84     ;
85         return EXIT_FAILURE;
86     }
87     filebuf fb;
88     fb.open(string(argv[1]), ios::in);
89     istream in(&fb);
90     calc::Lexer lexer(&in);
91     calc::Parser parser(&lexer);
92     parser.parse();
93     fb.close();
94     return 0;
95 }

```



```

edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/ejemplo/
• src$ ./calculadora prueba
Terminal ( detectado
Terminal numérico 9999 detectado
Terminal + detectado
Terminal numérico 1 detectado
Terminal ) detectado
Terminal * detectado
Terminal numérico 2 detectado
Salto de línea detectado
Análisis léxico y sintáctico terminado.
El valor de la expresión ya evaluada es: 20000
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/ejemplo/
• src$

```

Figura 1: Ejecución del ejemplo.

Para hacer el analizador sintáctico del lenguaje C₁, se usó la forma ascendente LALR(1) de Bison. No fue necesario eliminar la recursividad por la izquierda de la gramática, ya que Bison puede manejar producciones recursivas izquierdas de forma nativa.

1. **Archivo Flex (Ej. 3.a):** Se transcribió el archivo de Flex de la práctica 2, modificando las acciones de retorno para enviar los tokens generados automáticamente por la clase `Parser::token` de Bison. Se incluyeron los terminales de llaves `{}`.
2. **Gramática en Bison (Ej. 3.b):** Se transcribieron las producciones. Para resolver el problema de los desplazamiento/reducción por la la gramática ambigua de las expresiones, se utilizaron `%left MAS MENOS` y `%left MUL DIV`.

3. **Mensaje de error (Ej. 3.c):** Se cambió la función `error()` para que fuera más detallada al momento de fallar el análisis sintáctico.

Se presenta la implementación del analizador *LALR(1)* en el archivo `parser.yy`:

```
1 // ... (Declaraciones iniciales, namespaces y macros) ...
2
3 %token INT FLOAT IF ELSE WHILE
4 %token MAS MENOS MUL DIV ASIG PYC COMA
5 %token PARIZQ PARDER LLAVEIZQ LLAVEDER
6 %token IDENTIFICADOR NUMERO
7
8 %left MAS MENOS
9 %left MUL DIV
10
11 %start programa
12
13 %%
14
15 programa: declaraciones sentencias { cout << "Análisis concluido: Programa
16         valido." << endl; };
17
18 declaraciones: declaraciones declaracion | declaracion ;
19 declaracion: tipo lista_var PYC ;
20 tipo: INT | FLOAT ;
21 lista_var: lista_var COMA IDENTIFICADOR | IDENTIFICADOR ;
22 sentencias: sentencias sentencia | sentencia ;
23
24 sentencia: IDENTIFICADOR ASIG expresion PYC
25           | IF PARIZQ expresion PARDER LLAVEIZQ sentencias LLAVEDER ELSE
26             LLAVEIZQ sentencias LLAVEDER
27           | WHILE PARIZQ expresion PARDER LLAVEIZQ sentencias LLAVEDER
28           ;
29
30 expresion: expresion MAS expresion
31           | expresion MENOS expresion
32           | expresion MUL expresion
33           | expresion DIV expresion
34           | IDENTIFICADOR
35           | NUMERO
36           | PARIZQ expresion PARDER
37           ;
38
39 %%
40
41 void c1::Parser::error(const std::string& msg) {
42     std::cerr << "\n>>> ERROR SINTACTICO DETECTADO <<<" << '\n';
43     std::cerr << "Motivo reportado por Bison: " << msg << '\n';
44     std::cerr << "El analizador no pudo reducir la expresion." << '\n';
45 }
46
47 // ... (Funcion main) ...
```

4. **Proponer archivos de prueba (Ej. 3.d y 14).**

Archivos Válidos:

- prueba_valida_1: Declaraciones simples, múltiples y operaciones aritméticas respetando la precedencia LALR(1).

```
1 int a, b;  
2 float c;  
3 a = 10;  
4 b = 20;  
5 c = a + b * 2.5;
```

- prueba_valida_2: Estructuras while e if-else anidadas, respetando el uso de llaves de apertura y cierre.

```
1 int iterador;  
2 float acumulador;  
3 iterador = 5;  
4 acumulador = 0.0;  
5 while (iterador) {  
6     if (acumulador) {  
7         acumulador = acumulador + 1.5;  
8     } else {  
9         acumulador = acumulador - 1.5;  
10    }  
11    iterador = iterador - 1;  
12 }
```

Archivos Inválidos:

- prueba_invalida_1: Sin las llaves {}.

```
1 int x;  
2 x = 1;  
3 if (x)  
4     x = x + 1;  
5 else  
6     x = x - 1;
```

- prueba_invalida_2: Intentando poner una declaración (float y;) en la sección de sentencias.

```
1 int x;  
2 x = 10;  
3 float y;
```

Evidencia de Ejecución

La siguiente capturanuestra el analizador compilado reconociendo archivos válidos y atrapando errores en un archivos inválidos,.

```
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/C_1$ ./compilador prueba_valida_1
Análisis concluido: Programa valido segun la gramatica C_1.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/C_1$ ./compilador prueba_valida_2
Análisis concluido: Programa valido segun la gramatica C_1.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/C_1$ ./compilador prueba_invalida_1

>>> ERROR SINTACTICO DETECTADO <<<
Motivo reportado por Bison: syntax error
El analizador no pudo reducir la expresion con las reglas definidas en la gramatica.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/C_1$ ./compilador prueba_invalida_2

>>> ERROR SINTACTICO DETECTADO <<<
Motivo reportado por Bison: syntax error
El analizador no pudo reducir la expresion con las reglas definidas en la gramatica.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P4/C_1$
```

Figura 2: Ejecución del analizador LALR(1).

Extras

13. Documentar el código.

Se añadió documentación con comentarios tanto en el archivo `lexer.ll` como en `parser.yy`. En Bison, se documentó el uso de directivas de precedencia (`%left`) y se describió el axioma inicial. En Flex, se comentaron las expresiones regulares. Se puso el código fuente en este reporte.

Conclusiones Analíticas

El uso de generadores de analizadores sintácticos como Bison (LALR) nos da una ventaja en comparación a programar manual de analizadores predictivos de descenso recursivo (LL(1)). Durante esta práctica, se comprobó que Bison es capaz de manejar gramáticas con recursividad por la izquierda de manera nativa, sin la necesidad de poner variables auxiliares (como *declaraciones*’ o *sentencias*’).

La ambigüedad se simplificó con el uso de declaraciones de asociatividad (`%left`), lo que nos deja mantener las reglas de producción de manera más legible. La integración Flex-Bison demuestra ser una herramienta estándar y muy robusta para la construcción de un compilador.